

내비게이션 주행데이터를 이용한 목적지 **예상 도착시각** 개선

- 도메인 : 운송
- 목 표 : 자동차 내비게이션의 목적지 도착시간을 예측하는 인공지능모델 개발
- A사의 자동차 내비게이션이 경쟁사 상품에 비해 목적지 예상 도착시간(ETA : Estimated Time of Arrival)의 정확도가 떨어진다는 평가를 받으면서 판매량이 줄어들고 있다.
- 계속되는 판매량 부진을 만회하기 위해, A사 서비스개발팀장은 내비게이션 서비스를 하며 축적한 이용자의 주행데이터를 활용해서 '목적지 예상도착 시간'을 예측하는 인공지능 모델을 도입하려고 한다.
- A사는 브랜드파워가 있기 때문에, 인공지능을 활용해서 '목적지 예상 도착시간'의 정확도가 개선되면 자사 내비게이션의 판매량이 빠르게 회복될 것으로 기대하고 있다.

[유의사항]

- 각 문항의 답안은 반드시 '**#여기에 답안코드를 작성하고 실행하세요**', '**#여기에 답안을 입력하세요**' 등이 표시된 셀(cell)에 입력해야 합니다
- 제공된 시험문항 셀을 삭제하거나 답안 위치가 아닌 다른 셀에 답안코드를 작성 시 채점되지 않습니다
- 답안 작성 전에 문항에 제시된 가이드를 확인하세요
- 문항에 변수명이 제시된 경우 반드시 해당 변수명을 사용하세요
- 시험 중에는 상단의 '임시저장' 버튼을 수시로 클릭해 저장해주시고, 답안 제출시에는 '최종제출' 버튼을 클릭해주시기 바랍니다
- **시험 문항과 데이터 등을 무단으로 촬영/캡처, 복사/복제, 판매/재판매, 공개/전재, 공유/유포 할 경우 지식재산권 침해에 따른 법적 제재를 받을 수 있습니다**
- **오픈북은 허용된 사이트만 참고 가능하며 그 외 자료, 사이트 등을 참고하는 경우 부정행위로 간주됩니다**

[데이터 컬럼 설명 (데이터 파일명: signal_data.csv)]

- RID : 경로ID
- Time_Departure : 출발시각
- Time_Arrival : 도착시각
- Distance : 이동 거리, 단위 (m)
- Time_Driving : 실주행시간(초)
- Speed_Per_Hour : 평균시속
- Address1 : 주소1
- Address2 : 주소2
- Weekday : Time_Departure(출발시각)의 요일
- Hour : Time_Departure(출발시각)의 시각
- Day : Time_Departure(출발시각)의 날짜
- Signaltype : 경로의 신호등 갯수

<데이터 분석 (20점)>

1. pandas는 데이터 분석에 널리 사용되는 파이썬 라이브러리입니다. pandas를 사용할 수 있게 별칭(alias)을 pd로 해서 불러오세요.

In [1]: # (1) 여기에 답안코드를 작성하고 실행하세요.

```
import pandas as pd
```

2. AI 모델링을 위해 분석 및 처리할 데이터 파일을 읽어오려고 합니다. pandas로 데이터 파일을 읽어온 뒤, 데이터프레임 변수명 df에 할당하고 첫 4개 행을 출력하세요.

- 데이터프레임 변수명 : df
 - 데이터 파일명 : signal_data.csv
 - csv 파일은 본 문제/답안지와 동일한 경로에 있습니다
-

In [3]: # (2) 여기에 답안코드를 작성하고 실행하세요

```
df = pd.read_csv('./signal_data.csv')  
df.head(4)
```

Out[3]:

	RID	Time_Departure	Time_Arrival	Distance	Time_Driving	Speed_Per_Hour	Address1
0	router-84875df7fc-cmkz9-2-9617301-0	19:43:56	19:56:38.537000	3246.0	762.653992	15.322283	경기도
1	router-84875df7fc-sxcj-6-13366366-0	12:23:05	12:36:32.620000	3858.0	796.081970	17.446445	경기도
2	router-84875df7fc-52vz8-6-62202063-0	19:10:35	19:29:07.306000	4125.0	1112.468018	13.348698	경기도
3	router-84875df7fc-8n4sw-5-6914479-0	11:59:58	12:10:55.999000	3748.0	651.973022	20.695335	경기도

다음 문항을 풀기 전에 아래 코드를 실행하세요.

In [5]:

```
# 리눅스
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.font_manager as fm
plt.rc('font', family='NanumGothicCoding')

# 윈도우용으로 아래 한글 나오게 하기 위해 사용함.
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import matplotlib as mpl
import matplotlib.font_manager as fm
font_path = 'C:\\Windows\\Fonts\\gulim.ttc'
font = fm.FontProperties(fname=font_path).get_name()
mpl.rc('font', family=font)

import warnings
warnings.filterwarnings('ignore')
```

3. 시각화를 통해 네비게이션 목적지 주소1(Address1)의 분포를 파악하려 합니다.

- (3-1) 아래 가이드에 따라 seaborn 라이브러리를 활용해서 시각화하세요
 - 대상 데이터프레임 : df

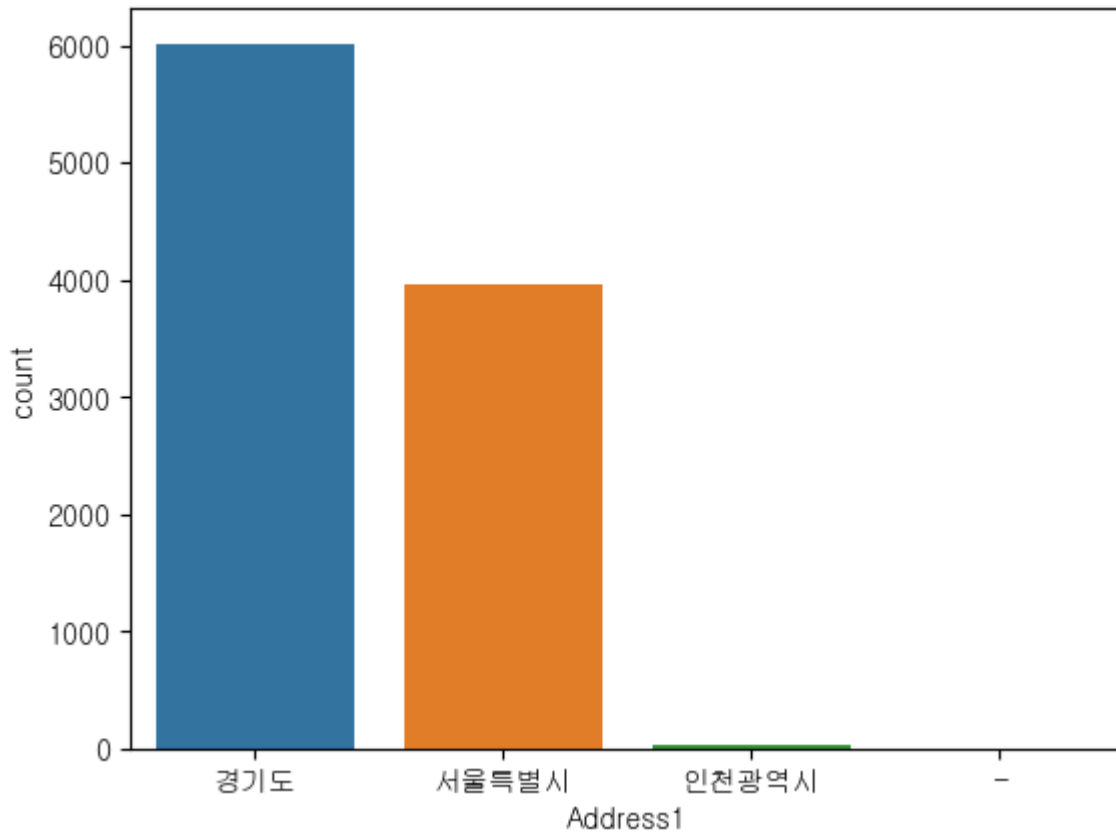
- 목적지 주소1의 분포를 보여주는 countplot 그래프를 그리세요

- (3-2) 출력된 그래프에서 내비게이션 이용자들이 가장 많이 찾는 목적지 주소1은 어디인가요?

In [6]: # (3-1) 여기에 답안코드를 작성하고 실행하세요

```
sns.countplot(data=df, x='Address1')
```

Out[6]: <Axes: xlabel='Address1', ylabel='count'>



In []: # (3-2) 여기에 답안을 입력하세요(실행 불필요)

경기도

4. 그래프를 통해 실주행시간(Time Driving)과 평균시속(Speed_Per_Hour) 컬럼의 분포와 관계를 확인하려 합니다.

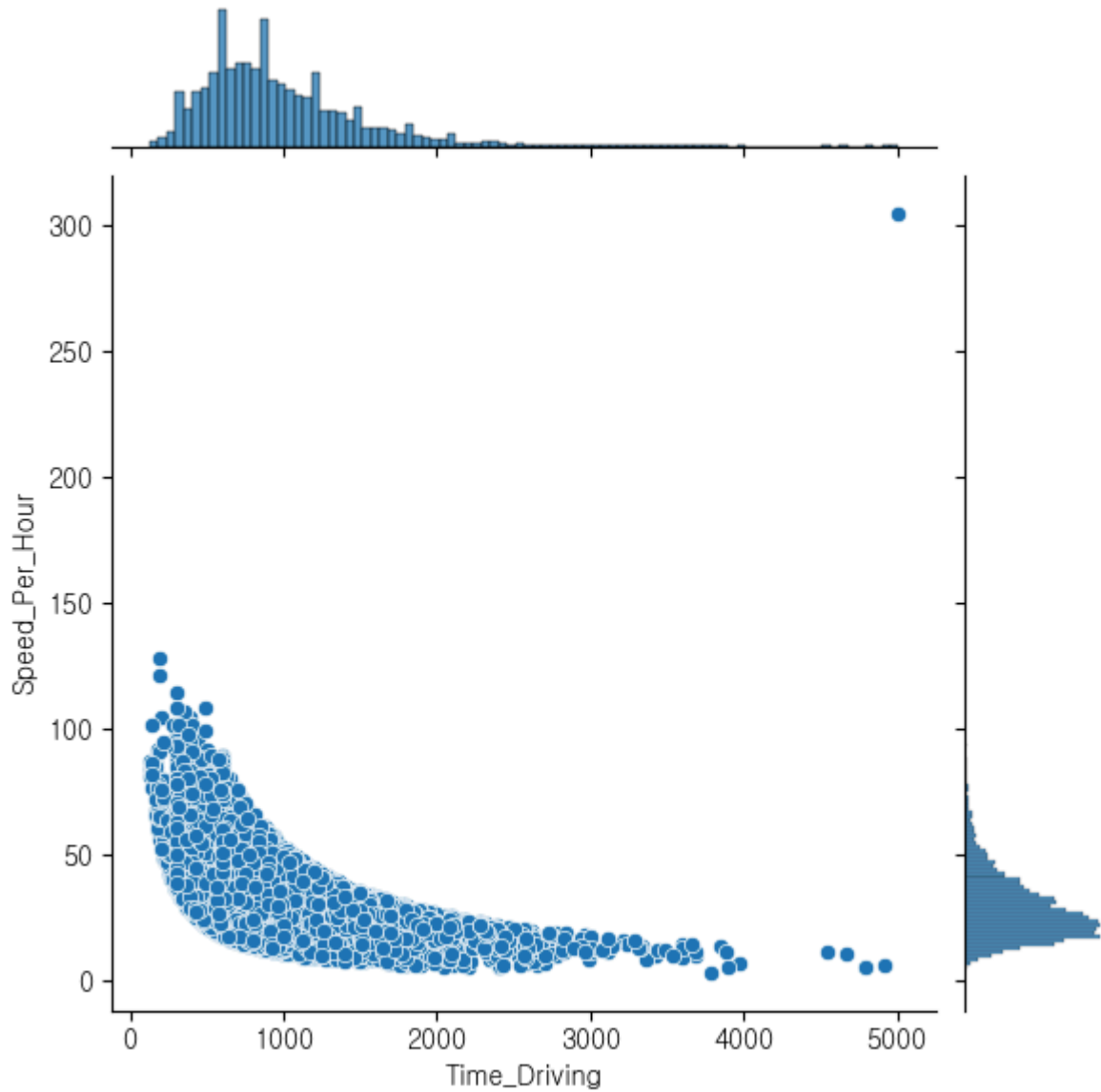
아래 가이드에 따라 jointplot으로 시각화하세요.

- 대상 데이터프레임 : df
- seaborn의 jointplot으로 두 컬럼 간의 관계를 그래프로 그리세요
- X축에는 실주행시간(Time_Driving)을, Y축에는 평균시속(Speed_Per_Hour)을 표시하세요

In [7]: # (4) 여기에 답안코드를 작성하고 실행하세요

```
sns.jointplot(data=df, x='Time_Driving', y='Speed_Per_Hour')
```

Out[7]: <seaborn.axisgrid.JointGrid at 0x1d1b290af90>



<데이터 전처리 (30점)>

5. jointplot으로 확인한 결과, 평균시속(Speed Per Hour) 컬럼에서 이상치가 발견되었습니다. AI모델링할때 부정적 영향이 없도록 아래 가이드에 따라 이상치를 처리하세요.

- 대상 데이터프레임 : df
- 평균시속이 300보다 크거나 같은 이상치가 있는 행(row)을 삭제하세요
- AI모델링에 불필요한 경로ID(RID) 컬럼을 삭제하세요
- 전처리를 적용한 데이터프레임은 df_temp라는 변수명으로 저장하세요

In [8]: # (6) 여기에 답안코드를 작성하고 실행하세요

```
df_temp = df[df.Speed_Per_Hour < 300]
df_temp.drop('RID', axis=1, inplace=True)
```

6. 데이터에 결측치가 있을 경우 시모델링의 성능이 저하될 수 있습니다. 아래 가이드에 따라 결측치를 확인하고 처리하세요.

- (6-1) 아래 가이드에 따라 결측치를 처리하고 검증하는데 에러가 나고 있습니다. 코드의 오류를 정정하세요
 - 대상 데이터프레임 : df_temp
 - 결측치 개수를 확인한 후에 결측치가 있는 행을 모두 삭제하세요
 - 전처리가 반영된 데이터프레임은 변수명 df_na에 저장하세요
 - 결측치가 사라졌는지 확인하세요
- (6-2) df_temp의 결측치 개수는 총 몇 개인가요?

In []: # (6-1) 여기에 코드의 오류를 정정하고 실행하세요

```
print( '결측치 처리전\n', df_temp.isnull().sum() )
df_na = df_temp.drop()
print( '\n결측치 처리후\n', df_na.isnull().total() )
```

In []: # (6-2) 여기에 답안을 입력하세요(실행 불필요)

In [10]: # [정답] (6-1) 여기에 코드의 오류를 정정하고 실행하세요

```
print( '결측치 처리전\n', df_temp.isnull().sum() )
df_na = df_temp.dropna()
print( '\n결측치 처리후\n', df_na.isnull().sum() )
```

```
결측치 처리전
Time_Departure    0
Time_Arrival      0
Distance          2
Time_Driving      0
Speed_Per_Hour    0
Address1          0
Address2          0
Weekday           0
Hour              0
Day               0
Signaltype        0
dtype: int64
```

```
결측치 처리후
Time_Departure    0
Time_Arrival      0
Distance          0
Time_Driving      0
Speed_Per_Hour    0
Address1          0
Address2          0
Weekday           0
Hour              0
Day               0
Signaltype        0
dtype: int64
```

```
In [ ]: # [정답] (6-2) 여기에 답안을 입력하세요(실행 불필요)
```

```
2
```

7. 원-핫 인코딩(One-Hot Encoding)은 범주형 데이터를 1과 0의 이진형 벡터로 변환하기 위해 사용하는 방법입니다.

아래 가이드에 따라 범주형 데이터를 이진형 벡터로 변환하는 코드를 완성하세요.

- 대상 데이터프레임 : df_na
 - 원-핫 인코딩 대상 : object형(문자형) 컬럼 전체
 - 원-핫인코딩이 반영된 결과를 데이터프레임 변수명 df_preset에 저장하세요
 - 원-핫인코딩한 결과, object형(문자형) 컬럼이 변환되었는지 info 함수로 검증하세요
-
- 아래 코드 셀(cell)에 있는 코드의 빈칸을 채우고 실행하세요
 - 빈칸에 들어갈 내용은 각 답안 셀(cell)에 입력하세요 (#7-1, #7-2)

```
In [ ]: # (코드 셀) 코드의 빈칸을 채우고 실행하세요
```

```
df_na.drop( ['Time_Departure', 'Time_Arrival'] , axis=1, inplace=True)
df_preset = pd. <#7-1> (data=df_na, <#7-2>=['Address1', 'Address2'] )
df_preset.info()
```

```
In [ ]: # (7-1) 여기에 답안을 입력하세요(실행 불필요)
```

```
In [ ]: # (7-2) 여기에 답안을 입력하세요(실행 불필요)
```

```
In [12]: # [정답] (코드 셀) 코드의 빈칸을 채우고 실행하세요
```

```
df_na.drop( ['Time_Departure', 'Time_Arrival'] , axis=1, inplace=True)  
df_preset = pd.get_dummies(data=df_na, columns=['Address1', 'Address2'] )  
df_preset.info()
```


<class 'pandas.core.frame.DataFrame'>

Index: 9997 entries, 0 to 9999

Data columns (total 82 columns):

#	Column	Non-Null Count	Dtype
0	Distance	9997 non-null	float64
1	Time_Driving	9997 non-null	float64
2	Speed_Per_Hour	9997 non-null	float64
3	Weekday	9997 non-null	int64
4	Hour	9997 non-null	int64
5	Day	9997 non-null	int64
6	Signaltype	9997 non-null	float64
7	Address1_-	9997 non-null	bool
8	Address1_경기도	9997 non-null	bool
9	Address1_서울특별시	9997 non-null	bool
10	Address1_인천광역시	9997 non-null	bool
11	Address2_가평군	9997 non-null	bool
12	Address2_강남구	9997 non-null	bool
13	Address2_강동구	9997 non-null	bool
14	Address2_강북구	9997 non-null	bool
15	Address2_강서구	9997 non-null	bool
16	Address2_계양구	9997 non-null	bool
17	Address2_고양시 덕양구	9997 non-null	bool
18	Address2_고양시 일산동구	9997 non-null	bool
19	Address2_고양시 일산서구	9997 non-null	bool
20	Address2_과천시	9997 non-null	bool
21	Address2_관악구	9997 non-null	bool
22	Address2_광명시	9997 non-null	bool
23	Address2_광주시	9997 non-null	bool
24	Address2_광진구	9997 non-null	bool
25	Address2_구로구	9997 non-null	bool
26	Address2_구리시	9997 non-null	bool
27	Address2_군포시	9997 non-null	bool
28	Address2_금천구	9997 non-null	bool
29	Address2_김포시	9997 non-null	bool
30	Address2_남동구	9997 non-null	bool
31	Address2_남양주시	9997 non-null	bool
32	Address2_노원구	9997 non-null	bool
33	Address2_도봉구	9997 non-null	bool
34	Address2_동대문구	9997 non-null	bool
35	Address2_동두천시	9997 non-null	bool
36	Address2_동작구	9997 non-null	bool
37	Address2_마포구	9997 non-null	bool
38	Address2_부천시	9997 non-null	bool
39	Address2_부평구	9997 non-null	bool
40	Address2_서구	9997 non-null	bool
41	Address2_서대문구	9997 non-null	bool
42	Address2_서초구	9997 non-null	bool
43	Address2_성남시 분당구	9997 non-null	bool
44	Address2_성남시 수정구	9997 non-null	bool
45	Address2_성남시 중원구	9997 non-null	bool
46	Address2_성동구	9997 non-null	bool
47	Address2_성북구	9997 non-null	bool
48	Address2_송파구	9997 non-null	bool
49	Address2_수원시 권선구	9997 non-null	bool
50	Address2_수원시 영통구	9997 non-null	bool
51	Address2_수원시 장안구	9997 non-null	bool
52	Address2_수원시 팔달구	9997 non-null	bool
53	Address2_시흥시	9997 non-null	bool
54	Address2_안산시 단원구	9997 non-null	bool
55	Address2_안산시 상록구	9997 non-null	bool
56	Address2_안성시	9997 non-null	bool
57	Address2_안양시 동안구	9997 non-null	bool

```

58 Address2_안양시 만안구      9997 non-null    bool
59 Address2_양주시           9997 non-null    bool
60 Address2_양천구           9997 non-null    bool
61 Address2_양평군           9997 non-null    bool
62 Address2_여주시           9997 non-null    bool
63 Address2_연천군           9997 non-null    bool
64 Address2_영등포구         9997 non-null    bool
65 Address2_오산시           9997 non-null    bool
66 Address2_용산구           9997 non-null    bool
67 Address2_용인시 기흥구     9997 non-null    bool
68 Address2_용인시 수지구     9997 non-null    bool
69 Address2_용인시 처인구     9997 non-null    bool
70 Address2_은평구           9997 non-null    bool
71 Address2_의왕시           9997 non-null    bool
72 Address2_의정부시         9997 non-null    bool
73 Address2_이천시           9997 non-null    bool
74 Address2_종로구           9997 non-null    bool
75 Address2_중구             9997 non-null    bool
76 Address2_중랑구           9997 non-null    bool
77 Address2_파주시           9997 non-null    bool
78 Address2_평택시           9997 non-null    bool
79 Address2_포천시           9997 non-null    bool
80 Address2_하남시           9997 non-null    bool
81 Address2_화성시           9997 non-null    bool

```

dtypes: bool(75), float64(4), int64(3)

memory usage: 1.3 MB

In []: # [정답] (7-1) 여기에 답안을 입력하세요(실행 불필요)

```
get_dummies
```

In []: # [정답] (7-2) 여기에 답안을 입력하세요(실행 불필요)

```
columns
```

8. 모델링후에 모델의 성능을 평가할 수 있도록 훈련과 검증 각각에 사용할 데이터셋을 분리하려고 합니다.

실주행시간(Time Driving) 컬럼을 레이블값 y로, 나머지 컬럼을 피쳐값 X로 할당한 후 훈련데이터셋과 검증데이터셋으로 분리하세요.

- 대상 데이터프레임 : df_preset
- 훈련과 검증데이터셋 분리
 - 훈련데이터셋 레이블 : y_train, 훈련데이터셋 피쳐: X_train
 - 검증데이터셋 레이블 : y_valid, 검증데이터셋 피쳐: X_valid
 - 훈련데이터셋과 검증데이터셋 비율은 80:20으로 설정하세요
 - 난수 시드는 42로 설정하세요
 - scikit-learn의 train_test_split 함수를 활용하세요

In [13]: # (8) 여기에 답안코드를 작성하고 실행하세요

```

X = df_preset.drop('Time_Driving', axis=1)
y = df_preset['Time_Driving']

```

```
from sklearn.model_selection import train_test_split
X_train, X_valid, y_train, y_valid = train_test_split(X, y, test_size=0.2, random_state=42)
```

9. 스케일링을 통해 데이터 이상치의 영향도를 낮추려고 합니다.

- (9-1) 아래 가이드에 따라 스케일링 처리하는데 에러가 나고 있습니다. 코드의 오류를 정정하세요
 - sklearn.preprocessing에서 RobustScaler 함수를 import하세요
 - RobustScaler 함수를 rs 변수에 지정하세요
 - 훈련데이터에 대해 RobustScaler를 학습시키고 변환을 적용하세요
 - 검증데이터에 학습된 스케일링 기준을 그대로 적용하세요
 - 스케일링된 검증데이터의 가장 큰 값을 찾고, 이를 반올림하세요
- (9-2) RobustScaler 적용 후 검증데이터의 최댓값을 반올림한 결과는 무엇입니까?

In []: # (9-1) 여기에 코드의 오류를 정정하고 실행하세요

```
from sklearn.preprocessing import RobustScaler
rs = RobustScaler()
X_train = rs.transform(X_train)
X_valid = rs.transform(X_train)
round(np.max(X_valid))
```

In []: # (9-2) 여기에 답안을 입력하세요(실행 불필요)

In [23]: # [정답] (9-1) 여기에 코드의 오류를 정정하고 실행하세요

```
from sklearn.preprocessing import RobustScaler
rs = RobustScaler()
X_train = rs.fit_transform(X_train)
X_valid = rs.transform(X_valid)
round( np.max(X_valid) )
```

Out[23]: 6

In []: # [정답] (9-2) 여기에 답안을 입력하세요(실행 불필요)

6

<AI 모델링(50점)>

10. 실주행시간(Time_Driving)을 예측하는 머신러닝 모델을 만들려고 합니다.

DecisionTree와 RandomForest는 여러 규칙을 순차적으로 적용하면서 독립변수 공간을 분할하는 모형으로서, 분류와 회귀 분석에 모두 사용될 수 있습니다.

- **(10-1) DecisionTree 모델의 하이퍼파라미터를 아래와 같이 구성하고, 변수에 저장하세요**
 - 트리의 최대 깊이 : 5
 - 노드를 분할하기 위한 최소한의 샘플 데이터수(min_samples_split) : 3
 - 난수 시드 : 120
 - Decision Tree 모델을 dt 변수에 저장하세요
 - 앞서 분리한 훈련데이터셋으로 fit 함수를 사용해 학습시키세요
- **(10-2) RandomForest 모델의 하이퍼파라미터를 아래와 같이 설정하고, 변수에 저장하세요**
 - 트리의 최대 깊이 : 5
 - 노드를 분할하기 위한 최소한의 샘플 데이터수(min_samples_split) : 3
 - 난수 시드 : 120
 - Random Forest 모델을 rf 변수에 저장하세요
 - 앞서 분리한 훈련데이터셋으로 fit 함수를 사용해 학습시키세요

In [15]: # (10-1) 여기에 답안코드를 작성하고 실행하세요

```
from sklearn.tree import DecisionTreeRegressor
dt = DecisionTreeRegressor(max_depth=5, min_samples_split=3, random_state=120)
dt.fit(X_train, y_train)
```

Out[15]: ▾ DecisionTreeRegressor

DecisionTreeRegressor(max_depth=5, min_samples_split=3, random_state=120)

In [16]: # (10-2) 여기에 답안코드를 작성하고 실행하세요

```
from sklearn.ensemble import RandomForestRegressor
rf = RandomForestRegressor(max_depth=5, min_samples_split=3, random_state=120)
rf.fit(X_train, y_train)
```

Out[16]: ▾ RandomForestRegressor

RandomForestRegressor(max_depth=5, min_samples_split=3, random_state=120)

11. 모델의 해석력과 개선 방향을 이해하기 위해 RandomForest 모델의 변수중요도를 파악하려고 합니다.

- **(11-1) 아래 가이드에 따라 모델의 변수중요도 파악하는데 에러가 나고 있습니다. 코드의 오류를 수정하세요**
 - 변수중요도 점수와 변수 이름을 매핑하여 DataFrame을 만드세요
 - 변수중요도 점수를 기준으로 내림차순 정렬하고 top 10개만 선정하세요.

- 변수중요도 점수를 막대 그래프로 시각화하세요

- (11-2) 출력된 그래프에서 가장 높은 중요도를 보이는 변수명을 쓰세요

In []: # (11-1) 여기에 코드의 오류를 정정하고 실행하세요

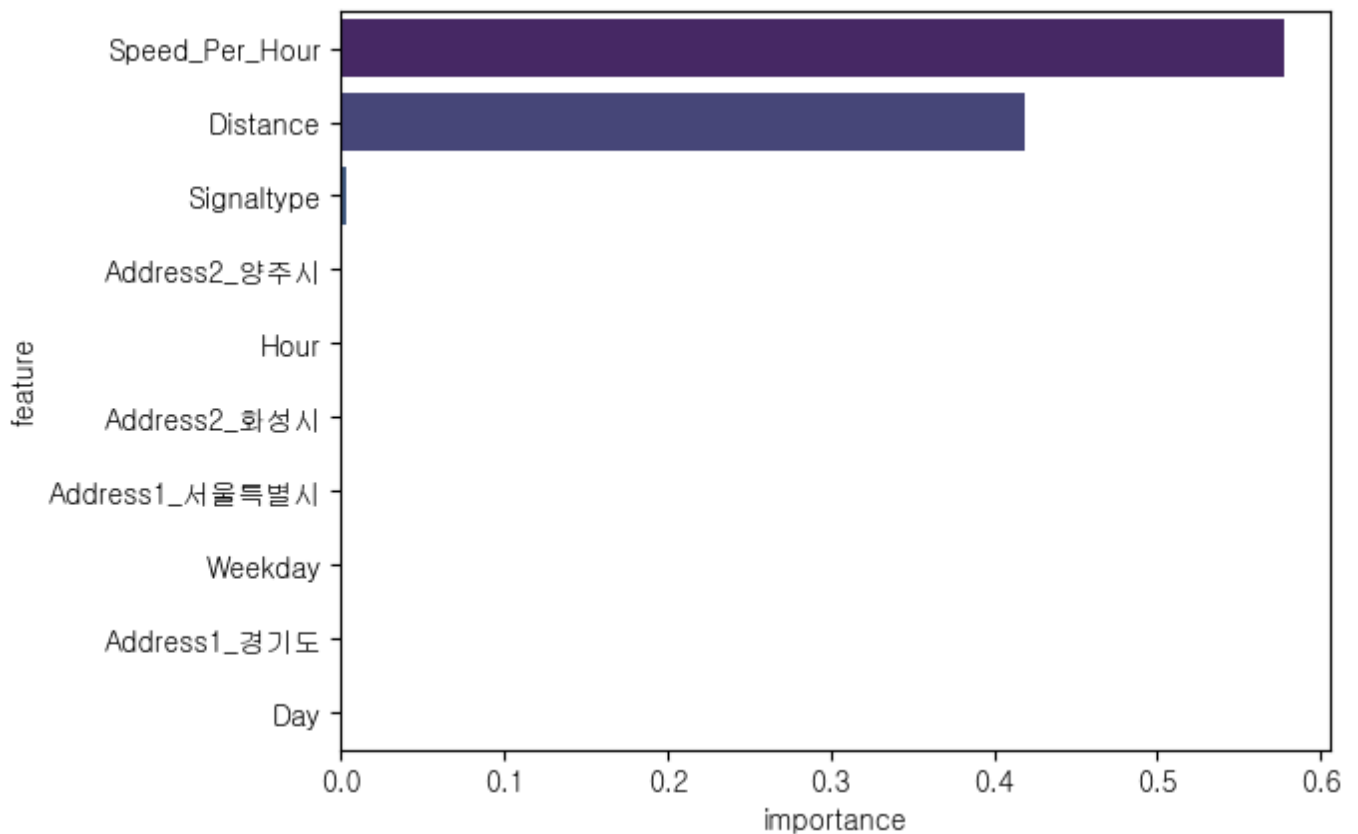
```
fi = pd.DataFrame({'feature': X.columns, 'importance': rf.importances})  
fi = fi.sort_index('importance', ascending=False)[:10]  
sns.barplot(x='importance', y='feature', data=fi, palette='viridis')
```

In []: # (11-2) 여기에 답안을 입력하세요(실행 불필요)

In [19]: # [정답] (11-1) 여기에 코드의 오류를 정정하고 실행하세요

```
fi = pd.DataFrame({'feature': X.columns, 'importance': rf.feature_importances_})  
fi = fi.sort_values('importance', ascending=False)[:10]  
sns.barplot(x='importance', y='feature', data=fi, palette='viridis')
```

Out[19]: <Axes: xlabel='importance', ylabel='feature'>



In []: # [정답] (11-2) 여기에 답안을 입력하세요(실행 불필요)

Speed_Per_Hour

12. 앞서 학습시킨 DecisionTree와 RandomForest 모델의 성능을 평가하려고 합니다.

- (12-1) 아래 가이드에 따라 각 모델의 MAE를 산출하려는데 에러가 나고 있습니다. 코드의 오류를 정정하세요

- 성능 평가는 검증데이터셋을 활용하세요
- 앞서 만든 DecisionTree 모델로 y값을 예측하고, 그 결과를 y_pred_dt 변수에 저장하세요
- 검증 정답(y_valid)과 예측값(y_pred_dt)의 MAE를 구하고 dt_mae 변수에 저장하세요
- 앞서 만든 RandomForest 모델로 y값을 예측하고, 그 결과를 y_pred_rf에 저장하세요
- 검증 정답(y_valid)과 예측값(y_pred_rf)의 MAE를 구하고 rf_mae 변수에 저장하세요
- 2개 모델의 MAE값을 출력하세요

- (12-2) 두 모델의 MAE 결과를 비교했을 때, 성능이 더 우수한 모델은 무엇인가요?

In []: # (12-1) 아래 코드의 오류를 정정하고 실행하세요

```
y_pred_dt = dt.predict(X_valid)
y_pred_rf = rf.predict(X_train)

from sklearn_metrics import mean.absolute

dt_mae = mean.absolute(y_valid, y_pred_rf)
rf_mae = mean.absolute(y_valid, y_pred_rf)
info(dt_mae, rf_mae)
```

In []: # (12-2) 여기에 답안을 입력하세요(실행 불필요)

In [41]: # (12-1) 아래 코드의 오류를 정정하고 실행하세요

```
y_pred_dt = dt.predict(X_valid)
y_pred_rf = rf.predict(X_valid)

from sklearn.metrics import mean_absolute_error

dt_mae = mean_absolute_error(y_valid, y_pred_dt)
rf_mae = mean_absolute_error(y_valid, y_pred_rf)
print(dt_mae, rf_mae)
```

109.70303411383863 59.168156727353164

In []: # (12-2) 여기에 답안을 입력하세요(실행 불필요)

```
RandomForest

randomforest (정답 인정)
Randomforest (정답 인정)
Random Forest (정답 인정)
```

다음 문항을 풀기 전에 아래 코드를 실행하세요.

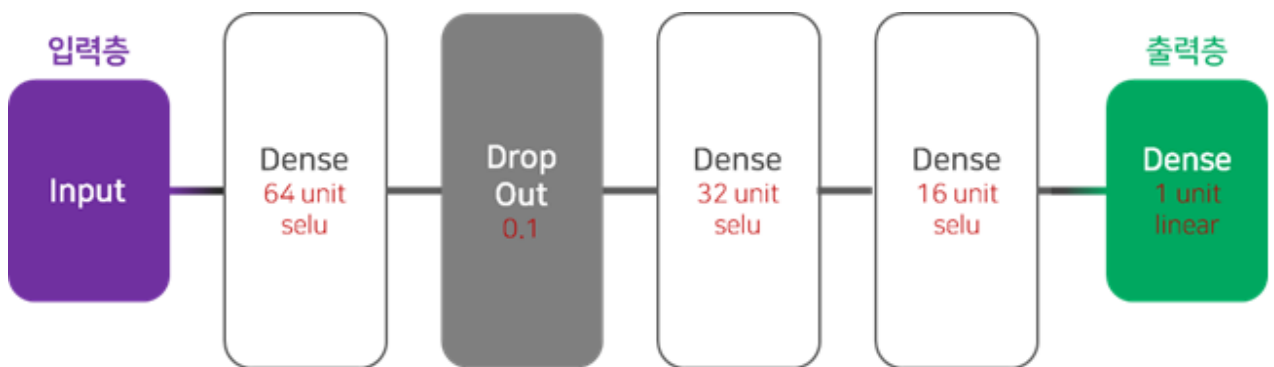
```
In [42]: import tensorflow as tf
from tensorflow.keras.models import Sequential, load_model
from tensorflow.keras.layers import Dense, Activation, Dropout, BatchNormalization
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
from tensorflow.keras.utils import to_categorical

tf.random.set_seed(1)
```

13. 실주행시간(Time_Driving)을 예측하는 딥러닝 모델을 만들려고 합니다.

아래 토폴로지 다이어그램과 가이드를 참고해서 모델링하고 학습을 수행하세요.

- tensorflow framework를 사용하고 하단의 토폴로지 그림과 동일한 딥러닝 모델을 구현하세요
- 히든 레이어의 activation 함수는 'selu'를, 마지막 아웃풋 레이어의 activation 함수는 'linear'를 사용하세요
- EarlyStopping 함수를 사용해서 9번의 epoch 동안 모니터링 지표(val_loss)가 향상되지 않을 경우 훈련을 중지하도록 설정하고 estop 변수에 저장하세요
- optimizer는 adam, metrics는 mse, loss는 mean_squared_error로 지정해서 모델을 컴파일하세요
- 다음 조건에 따라 모델을 학습하고 학습정보는 history 변수에 저장하세요
 - batch_size : 128
 - epoch : 50
- 안내된 내용 외 별도의 파라미터를 입력하지 마세요



In [43]: # (13) 여기에 답안코드를 작성하고 실행하세요

```
model = Sequential()
model.add(Dense(64, activation='selu', input_dim=X.shape[1]))
model.add(Dropout(0.1))
model.add(Dense(32, activation='selu'))
model.add(Dense(16, activation='selu'))
model.add(Dense(1, activation='linear'))

estop = EarlyStopping(monitor='val_loss', patience=9)
model.compile(optimizer='adam', loss='mean_squared_error', metrics = ['mse'])

history = model.fit(X_train, y_train, batch_size=128, epochs=50,
                    validation_data=(X_valid, y_valid), callbacks=[estop])
```

Epoch 1/50
63/63 ————— 2s 7ms/step - loss: 1174135.8750 - mse: 1174135.8750 - val_loss: 1115467.8750 - val_mse: 1115467.8750
Epoch 2/50
63/63 ————— 0s 4ms/step - loss: 1052029.6250 - mse: 1052029.6250 - val_loss: 854063.0625 - val_mse: 854063.0625
Epoch 3/50
63/63 ————— 0s 4ms/step - loss: 586404.0625 - mse: 586404.0625 - val_loss: 264958.7188 - val_mse: 264958.7188
Epoch 4/50
63/63 ————— 0s 4ms/step - loss: 171135.7344 - mse: 171135.7344 - val_loss: 120908.6562 - val_mse: 120908.6562
Epoch 5/50
63/63 ————— 0s 4ms/step - loss: 112200.3281 - mse: 112200.3281 - val_loss: 93287.1328 - val_mse: 93287.1328
Epoch 6/50
63/63 ————— 0s 4ms/step - loss: 88106.7812 - mse: 88106.7812 - val_loss: 74582.4531 - val_mse: 74582.4531
Epoch 7/50
63/63 ————— 0s 4ms/step - loss: 71806.4609 - mse: 71806.4609 - val_loss: 62164.4258 - val_mse: 62164.4258
Epoch 8/50
63/63 ————— 0s 4ms/step - loss: 59708.6680 - mse: 59708.6680 - val_loss: 54656.0391 - val_mse: 54656.0391
Epoch 9/50
63/63 ————— 0s 4ms/step - loss: 53254.5977 - mse: 53254.5977 - val_loss: 50480.8594 - val_mse: 50480.8594
Epoch 10/50
63/63 ————— 0s 4ms/step - loss: 49352.3555 - mse: 49352.3555 - val_loss: 48181.8086 - val_mse: 48181.8086
Epoch 11/50
63/63 ————— 0s 4ms/step - loss: 47255.3672 - mse: 47255.3672 - val_loss: 46801.6602 - val_mse: 46801.6602
Epoch 12/50
63/63 ————— 0s 4ms/step - loss: 45758.5586 - mse: 45758.5586 - val_loss: 45908.4648 - val_mse: 45908.4648
Epoch 13/50
63/63 ————— 0s 3ms/step - loss: 45058.9453 - mse: 45058.9453 - val_loss: 45318.2461 - val_mse: 45318.2461
Epoch 14/50
63/63 ————— 0s 4ms/step - loss: 44584.2539 - mse: 44584.2539 - val_loss: 44759.3672 - val_mse: 44759.3672
Epoch 15/50
63/63 ————— 0s 4ms/step - loss: 43770.8867 - mse: 43770.8867 - val_loss: 44370.1289 - val_mse: 44370.1289
Epoch 16/50
63/63 ————— 0s 3ms/step - loss: 43505.0312 - mse: 43505.0312 - val_loss: 43877.5000 - val_mse: 43877.5000
Epoch 17/50
63/63 ————— 0s 4ms/step - loss: 42762.8789 - mse: 42762.8789 - val_loss: 43438.7227 - val_mse: 43438.7227
Epoch 18/50
63/63 ————— 0s 6ms/step - loss: 42889.3516 - mse: 42889.3516 - val_loss: 42934.6211 - val_mse: 42934.6211
Epoch 19/50
63/63 ————— 0s 5ms/step - loss: 42611.0820 - mse: 42611.0820 - val_loss: 42499.3750 - val_mse: 42499.3750
Epoch 20/50
63/63 ————— 0s 4ms/step - loss: 40730.9922 - mse: 40730.9922 - val_loss: 41903.6641 - val_mse: 41903.6641
Epoch 21/50
63/63 ————— 0s 4ms/step - loss: 40409.0742 - mse: 40409.0742 - val_loss: 41263.5117 - val_mse: 41263.5117

Epoch 22/50
63/63 ————— 0s 5ms/step - loss: 40468.2812 - mse: 40468.2812 - val_loss: 40655.7812 - val_mse: 40655.7812
Epoch 23/50
63/63 ————— 0s 4ms/step - loss: 40223.7891 - mse: 40223.7891 - val_loss: 39925.8203 - val_mse: 39925.8203
Epoch 24/50
63/63 ————— 0s 4ms/step - loss: 38950.8164 - mse: 38950.8164 - val_loss: 39160.1367 - val_mse: 39160.1367
Epoch 25/50
63/63 ————— 0s 6ms/step - loss: 38295.3398 - mse: 38295.3398 - val_loss: 38274.0078 - val_mse: 38274.0078
Epoch 26/50
63/63 ————— 1s 8ms/step - loss: 37685.4180 - mse: 37685.4180 - val_loss: 37291.8320 - val_mse: 37291.8320
Epoch 27/50
63/63 ————— 0s 7ms/step - loss: 36229.3828 - mse: 36229.3828 - val_loss: 36348.6055 - val_mse: 36348.6055
Epoch 28/50
63/63 ————— 0s 7ms/step - loss: 35249.2656 - mse: 35249.2656 - val_loss: 35348.0039 - val_mse: 35348.0039
Epoch 29/50
63/63 ————— 0s 5ms/step - loss: 34233.5586 - mse: 34233.5586 - val_loss: 34242.0195 - val_mse: 34242.0195
Epoch 30/50
63/63 ————— 1s 4ms/step - loss: 33449.7070 - mse: 33449.7070 - val_loss: 33130.4570 - val_mse: 33130.4570
Epoch 31/50
63/63 ————— 0s 5ms/step - loss: 32389.1230 - mse: 32389.1230 - val_loss: 31881.2227 - val_mse: 31881.2227
Epoch 32/50
63/63 ————— 0s 5ms/step - loss: 31709.7207 - mse: 31709.7207 - val_loss: 30647.5586 - val_mse: 30647.5586
Epoch 33/50
63/63 ————— 1s 4ms/step - loss: 30400.5078 - mse: 30400.5078 - val_loss: 29308.3027 - val_mse: 29308.3027
Epoch 34/50
63/63 ————— 0s 5ms/step - loss: 29140.0762 - mse: 29140.0762 - val_loss: 27847.4492 - val_mse: 27847.4492
Epoch 35/50
63/63 ————— 0s 5ms/step - loss: 27825.5801 - mse: 27825.5801 - val_loss: 26401.8398 - val_mse: 26401.8418
Epoch 36/50
63/63 ————— 0s 4ms/step - loss: 26393.8613 - mse: 26393.8613 - val_loss: 24855.2598 - val_mse: 24855.2598
Epoch 37/50
63/63 ————— 0s 3ms/step - loss: 24604.3633 - mse: 24604.3633 - val_loss: 23180.8926 - val_mse: 23180.8926
Epoch 38/50
63/63 ————— 0s 4ms/step - loss: 24127.6621 - mse: 24127.6621 - val_loss: 21607.9023 - val_mse: 21607.9023
Epoch 39/50
63/63 ————— 0s 4ms/step - loss: 21900.8281 - mse: 21900.8281 - val_loss: 20031.7461 - val_mse: 20031.7461
Epoch 40/50
63/63 ————— 0s 5ms/step - loss: 20410.5410 - mse: 20410.5410 - val_loss: 18625.5020 - val_mse: 18625.5020
Epoch 41/50
63/63 ————— 0s 5ms/step - loss: 19125.6133 - mse: 19125.6133 - val_loss: 17399.6895 - val_mse: 17399.6895
Epoch 42/50
63/63 ————— 0s 5ms/step - loss: 18234.8379 - mse: 18234.8379 - val_loss: 16295.6230 - val_mse: 16295.6230

```
Epoch 43/50
63/63 ----- 0s 5ms/step - loss: 17221.0625 - mse: 17221.0625 - val_loss: 15313.8672 - val_mse: 15313.8672
Epoch 44/50
63/63 ----- 0s 4ms/step - loss: 15932.3428 - mse: 15932.3428 - val_loss: 14419.1846 - val_mse: 14419.1846
Epoch 45/50
63/63 ----- 0s 4ms/step - loss: 15455.4775 - mse: 15455.4775 - val_loss: 13582.6797 - val_mse: 13582.6797
Epoch 46/50
63/63 ----- 0s 5ms/step - loss: 14635.0654 - mse: 14635.0654 - val_loss: 12830.0811 - val_mse: 12830.0811
Epoch 47/50
63/63 ----- 0s 4ms/step - loss: 14095.8584 - mse: 14095.8584 - val_loss: 12169.4570 - val_mse: 12169.4570
Epoch 48/50
63/63 ----- 0s 4ms/step - loss: 13190.5195 - mse: 13190.5195 - val_loss: 11577.7861 - val_mse: 11577.7861
Epoch 49/50
63/63 ----- 0s 4ms/step - loss: 12529.9688 - mse: 12529.9688 - val_loss: 11086.2959 - val_mse: 11086.2959
Epoch 50/50
63/63 ----- 0s 5ms/step - loss: 11976.4600 - mse: 11976.4600 - val_loss: 10456.8398 - val_mse: 10456.8398
```

14. 앞서 만든 딥러닝 모델의 일반화 성능(Generalization Performance)을 평가하려고 합니다. matplotlib 라이브러리 활용해서, 학습 MSE와 검증 MSE의 변화를 그래프로 시각화하세요.

- 아래 가이드에 따라 MSE 변화를 시각화하는 코드를 완성하세요
 - 1개의 그래프에 학습 MSE와 검증 MSE 2가지를 모두 표시하세요
 - 위 2가지 각각의 범례를 'mse', 'val_mse'로 표시하세요
 - 그래프의 타이틀은 'Model MSE'로 표시하세요
 - X축에는 'Epochs'라고 표시하고 Y축에는 'MSE'라고 표시하세요
- 아래 코드 셀(cell)에 있는 코드의 빈칸을 채우고 실행하세요
- 빈칸에 들어갈 내용은 각 답안 셀(cell)에 입력하세요 (#14-1, #14-2)

In []: # (코드 셀) 코드의 빈칸을 채우고 실행하세요

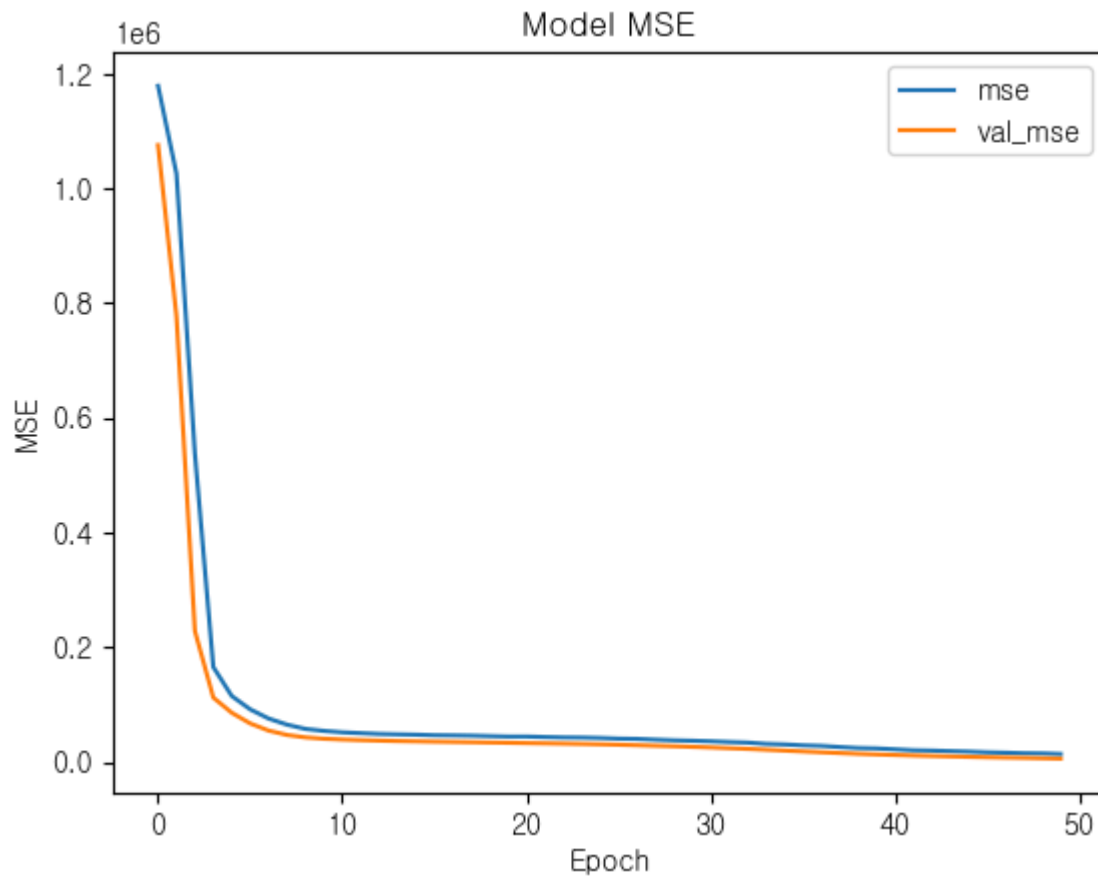
```
plt.plot(<#14-1>["mse"])
plt.plot(<#14-1>["val_mse"])
plt.title("Model MSE")
plt.xlabel("Epoch")
plt.ylabel("MSE")
plt.<#14-2>(['mse', 'val_mse'])
plt.show()
```

In []: # (14-1) 여기에 답안을 입력하세요(실행 불필요)

In []: # (14-2) 여기에 답안을 입력하세요(실행 불필요)

In [17]: # (코드 셀) 코드의 빈칸을 채우고 실행하세요

```
plt.plot(history.history["mse"])
plt.plot(history.history["val_mse"])
plt.title("Model MSE")
plt.xlabel("Epoch")
plt.ylabel("MSE")
plt.legend(['mse', 'val_mse'])
plt.show()
```



In []: # (14-1) 여기에 답안을 입력하세요 (실행 불필요)

```
history.history
```

In []: # (14-2) 여기에 답안을 입력하세요 (실행 불필요)

```
legend
```

모든 문항이 완료되었습니다. 수고하셨습니다.

'최종제출 및 종료'를 클릭하셔서 답안을 제출해 주시기 바랍니다.